

Counterfactual Computer Science

Soumadeep Ghosh

Kolkata, India

Abstract

Classical computer science studies actual computation: algorithms, programs, machines, languages, systems, data, and learned models as they execute in a given environment. This paper proposes *Counterfactual Computer Science* as a prospective branch of computer science in which the primary object is not a single computation, but a structured family of unrealized computations near the actual one. The field studies what programs, machines, networks, and artificial intelligence systems would have done under alternative inputs, hardware states, resource bounds, execution histories, data-generating processes, adversaries, and institutional environments. We introduce counterfactual computational worlds, counterfactual program bundles, counterfactual correctness, counterfactual equivalence, counterfactual complexity, robustness, debugging, security, statistics, data science, and machine learning. The paper gives definitions, axioms, propositions, proofs, algorithms, tables, and PGF/TikZ diagrams suitable for a first formal account of the field.

The paper ends with “The End”

Contents

1	Introduction	2
2	Classical Computation and Counterfactual Computation	2
3	Computational Worlds	3
4	Counterfactual Program Bundles	3
5	Foundational Axiom	4
6	Counterfactual Correctness	4
7	Counterfactual Equivalence	5
8	Counterfactual Robustness	5
9	Counterfactual Complexity Theory	6
10	Counterfactual Debugging	6
11	Counterfactual Security	7
12	Counterfactual Statistics and Data Science	7
13	Counterfactual Machine Learning	8
14	Counterfactual Operating Systems and Distributed Systems	9

15	Counterfactual Programming Languages	10
16	A General Architecture for Counterfactual Analysis	10
17	Research Program	11
18	Limitations and Boundary Conditions	11
19	Conclusion	11

List of Figures

1	The actual computational world ω_0 and six nearby counterfactual worlds contained inside the neighborhood $\mathcal{N}_r(\omega_0)$. Each arrow denotes a distinct admissible perturbation.	3
2	A world-indexed computational pipeline.	9
3	A generic architecture for Counterfactual Computer Science.	10

List of Tables

1	Classical and counterfactual forms of computer-scientific questions.	2
2	Classical and counterfactual statistical objects.	8
3	A possible research map for Counterfactual Computer Science.	11

The following space was deliberately left blank.

1 Introduction

The usual question of computation is

$$P(x) = y,$$

where program P maps input x to output y . Counterfactual Computer Science asks a stronger question:

$$P_\omega(x_\omega) = y_\omega \quad \text{for } \omega \in \Omega,$$

where Ω is a space of possible computational worlds. A world may vary the input, program, compiler, hardware, scheduler, operating system, network, training data, adversary, random seed, resource constraint, or environment.

The central claim is that a computation is not fully characterized by what it did. It is characterized by what it did relative to what it would have done under nearby alternatives. Thus, the object of study is not merely

$$C = (P, x, M, R, E, H),$$

but the neighborhood

$$\mathcal{N}(C) = \{C_\omega : \omega \in \Omega, d_\Omega(\omega, \omega_0) \leq r\}.$$

This paper develops a formal foundation for this proposed branch.

2 Classical Computation and Counterfactual Computation

Classical computer science contains fragments of counterfactual reasoning: program verification, model checking, simulation, randomized algorithms, fault tolerance, causal inference, adversarial machine learning, distributed systems, and complexity theory. Counterfactual Computer Science treats these fragments as manifestations of one common object: the unrealized computation.

Area	Classical question	Counterfactual question
Algorithms	What is the runtime on input x ?	What would the runtime be under alternate worlds?
Verification	Does P satisfy specification S ?	Does P_ω satisfy S_ω for all relevant ω ?
Complexity	Is a language in P , NP , or BPP ?	Is it efficiently solvable across structured world perturbations?
Debugging	Why did failure occur?	What smallest world-change would have prevented failure?
Security	Can an adversary break the system?	Across which adversary-worlds is the system secure?
Machine learning	What does the model predict?	What would it predict under alternate data, seeds, or deployment contexts?
Distributed systems	What happened in this execution trace?	What would have happened under alternate schedules or message delays?

Table 1: Classical and counterfactual forms of computer-scientific questions.

3 Computational Worlds

Definition 1 (Computational world). *A computational world is a tuple*

$$\omega = (X_\omega, Y_\omega, P_\omega, M_\omega, R_\omega, E_\omega, H_\omega, A_\omega, \mu_\omega),$$

where X_ω is the input space, Y_ω is the output space, P_ω is a program or model, M_ω is a machine or architecture, R_ω is a vector of resource constraints, E_ω is the environment, H_ω is an execution history, A_ω is an adversary or perturbation process, and μ_ω is a probability law over events in that world.

Definition 2 (Counterfactual world space). *A counterfactual world space is a triple*

$$(\Omega, d_\Omega, \omega_0),$$

where Ω is a set of computational worlds, d_Ω is a distance, divergence, preorder, or graph metric on worlds, and $\omega_0 \in \Omega$ is the actual world.

Definition 3 (Counterfactual neighborhood). *For radius $r \geq 0$, the counterfactual neighborhood of the actual world is*

$$\mathcal{N}_r(\omega_0) = \{\omega \in \Omega : d_\Omega(\omega, \omega_0) \leq r\}.$$

The value of r determines the epistemic horizon of the analysis. Small r studies nearby bugs, seeds, schedules, and faults. Large r studies different platforms, institutions, adversaries, data-generating processes, or technological regimes.

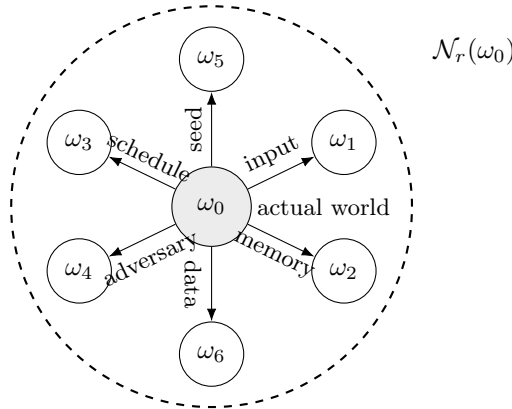


Figure 1: The actual computational world ω_0 and six nearby counterfactual worlds contained inside the neighborhood $\mathcal{N}_r(\omega_0)$. Each arrow denotes a distinct admissible perturbation.

4 Counterfactual Program Bundles

Definition 4 (Program bundle). *A counterfactual program bundle is a family*

$$\mathbf{P} = \{P_\omega : X_\omega \rightarrow Y_\omega\}_{\omega \in \Omega}.$$

The actual program is P_{ω_0} . The counterfactual programs P_ω , $\omega \neq \omega_0$, encode alternative execution conditions.

Definition 5 (Outcome profile). *Given a world-indexed input family*

$$\mathbf{x} = \{x_\omega \in X_\omega\}_{\omega \in \Omega},$$

the counterfactual outcome profile of $\mathbf{P}$ is

$$\mathbf{y}(\mathbf{P}, \mathbf{x}) = \{P_\omega(x_\omega)\}_{\omega \in \Omega}.$$

Example 1 (Loan decision profile). *If an AI model rejects a borrower in the actual world but accepts the borrower under corrected income data, alternate credit-bureau data, and a different training sample, the relevant object is not the single decision 0, but the profile*

$$(0, 1, 1, 0, 1).$$

This profile reveals fragility, data-dependence, and possible arbitrariness.

5 Foundational Axiom

Axiom 1 (Counterfactual Computation Axiom). *For every computation C , there exists a structured neighborhood of unrealized computations $\mathcal{N}(C)$. The meaning, reliability, risk, and interpretability of C are functions of both the actual computation and its counterfactual neighborhood:*

$$\text{Meaning}(C) = F(C, \mathcal{N}(C)).$$

This axiom does not deny actual execution. Rather, it asserts that actual execution is an insufficient statistic for computational significance.

6 Counterfactual Correctness

Definition 6 (World-indexed specification). *A world-indexed specification is a family*

$$\mathbf{S} = \{S_\omega \subseteq X_\omega \times Y_\omega\}_{\omega \in \Omega}.$$

The pair $(x, y) \in S_\omega$ means that output y is acceptable for input x in world ω .

Definition 7 (Counterfactual correctness). *A program bundle $\mathbf{P}$ is counterfactually correct over $\Omega' \subseteq \Omega$ with respect to $\mathbf{S}$ if*

$$\forall \omega \in \Omega', \forall x \in X_\omega, \quad (x, P_\omega(x)) \in S_\omega.$$

Definition 8 (ϵ -counterfactual correctness). *Suppose each world has a loss function*

$$\ell_\omega : Y_\omega \times Y_\omega \rightarrow \mathbb{R}_{\geq 0}.$$

A bundle $\mathbf{P}$ is ϵ -counterfactually correct if

$$\sup_{\omega \in \Omega'} \sup_{x \in X_\omega} \ell_\omega(P_\omega(x), f_\omega(x)) \leq \epsilon.$$

Proposition 1 (Classical correctness as a special case). *If $\Omega' = \{\omega_0\}$, then counterfactual correctness reduces to classical correctness in the actual world.*

Proof. With $\Omega' = \{\omega_0\}$, the universal quantifier over worlds contains only ω_0 . Hence the definition becomes

$$\forall x \in X_{\omega_0}, \quad (x, P_{\omega_0}(x)) \in S_{\omega_0},$$

which is exactly ordinary partial or total correctness, depending on whether termination is included in S_{ω_0} . $\square$

The following space was deliberately left blank.

7 Counterfactual Equivalence

Definition 9 (Counterfactual equivalence). *Two program bundles $\mathbf{P}$ and $\mathbf{Q}$ are counterfactually equivalent over $\Omega' \subseteq \Omega$, written*

$$\mathbf{P} \equiv_{\Omega'} \mathbf{Q},$$

if

$$\forall \omega \in \Omega', \forall x \in X_\omega, \quad P_\omega(x) = Q_\omega(x).$$

Theorem 1 (Equivalence relation). *Counterfactual equivalence $\equiv_{\Omega'}$ is an equivalence relation.*

Proof. Reflexivity follows because $P_\omega(x) = P_\omega(x)$. Symmetry follows because equality is symmetric. Transitivity follows because if $P_\omega(x) = Q_\omega(x)$ and $Q_\omega(x) = R_\omega(x)$, then $P_\omega(x) = R_\omega(x)$. Since these statements hold for every $\omega \in \Omega'$ and every $x \in X_\omega$, the relation is reflexive, symmetric, and transitive. $\square$

8 Counterfactual Robustness

Definition 10 (Counterfactual robustness modulus). *Let d_Y be an output distance. For a program bundle $\mathbf{P}$, define*

$$\rho_r(\mathbf{P}) = \sup_{\substack{\omega, \omega' \in \mathcal{N}_r(\omega_0) \\ x_\omega \in X_\omega, x_{\omega'} \in X_{\omega'}}} \frac{d_Y(P_\omega(x_\omega), P_{\omega'}(x_{\omega'}))}{d_\Omega(\omega, \omega') + d_X(x_\omega, x_{\omega'})},$$

where the ratio is interpreted over nonzero denominators.

Low $\rho_r(\mathbf{P})$ indicates that nearby computational worlds induce nearby outputs. High $\rho_r(\mathbf{P})$ indicates brittleness.

Theorem 2 (Composition of counterfactual robustness). *Let $\mathbf{P}$ and $\mathbf{Q}$ be bundles such that*

$$P_\omega : X_\omega \rightarrow Z_\omega, \quad Q_\omega : Z_\omega \rightarrow Y_\omega.$$

If $\mathbf{P}$ is L_P -robust and $\mathbf{Q}$ is L_Q -robust on $\mathcal{N}_r(\omega_0)$, then the composed bundle

$$(\mathbf{Q} \circ \mathbf{P})_\omega = Q_\omega \circ P_\omega$$

is $L_Q L_P$ -robust, up to the metric distortion induced by the intermediate spaces Z_ω .

Proof. For nearby worlds ω, ω' , let

$$z_\omega = P_\omega(x_\omega), \quad z_{\omega'} = P_{\omega'}(x_{\omega'}).$$

By robustness of $\mathbf{Q}$,

$$d_Y(Q_\omega(z_\omega), Q_{\omega'}(z_{\omega'})) \leq L_Q d_Z(z_\omega, z_{\omega'}).$$

By robustness of $\mathbf{P}$,

$$d_Z(z_\omega, z_{\omega'}) \leq L_P [d_\Omega(\omega, \omega') + d_X(x_\omega, x_{\omega'})].$$

Combining the two inequalities gives

$$d_Y((Q_\omega \circ P_\omega)(x_\omega), (Q_{\omega'} \circ P_{\omega'})(x_{\omega'})) \leq L_Q L_P [d_\Omega(\omega, \omega') + d_X(x_\omega, x_{\omega'})].$$

Thus the composed bundle is robust with constant $L_Q L_P$, modulo any world-dependent distortion in the definition of d_Z . $\square$

9 Counterfactual Complexity Theory

Definition 11 (Perturbation system). *A perturbation system is a sequence*

$$\Pi = \{(\Omega_n, d_n, \mu_n)\}_{n \in \mathbb{N}},$$

where Ω_n is the set of worlds relevant to inputs of size n , d_n is a world distance, and μ_n is a distribution over worlds.

Definition 12 (Counterfactual polynomial time). *A world-indexed language bundle*

$$\mathbf{L} = \{L_\omega \subseteq \{0, 1\}^*\}_{\omega \in \Omega}$$

belongs to $\mathbf{CFP}_\Pi$ if there exist a program bundle $\mathbf{P}$ and a constant k such that for every n , every $\omega \in \Omega_n$, and every $x \in \{0, 1\}^n$,

$$P_\omega(x) = \mathbf{1}\{x \in L_\omega\},$$

and

$$T(P_\omega, x) \leq n^k.$$

Definition 13 (Counterfactual randomized polynomial time). *A language bundle $\mathbf{L}$ belongs to $\mathbf{CFBPP}_\Pi$ if there exist a randomized program bundle $\mathbf{P}$, constants k and $\delta < 1/3$, such that for all n , all $\omega \in \Omega_n$, and all $x \in \{0, 1\}^n$,*

$$\mathbb{P}[P_\omega(x) = \mathbf{1}\{x \in L_\omega\}] \geq 1 - \delta,$$

with runtime bounded by n^k .

Theorem 3 (Classical inclusion). *If $\Omega_n = \{\omega_0\}$ for all n , then*

$$\mathbf{CFP}_\Pi = P$$

for the actual language L_{ω_0} .

Proof. When each Ω_n contains only ω_0 , the counterfactual universal condition over worlds disappears. The definition requires exactly one polynomial-time algorithm deciding L_{ω_0} . This is the classical definition of P . $\square$

Proposition 2 (Worst-world strengthening). *For any perturbation system Π , membership in $\mathbf{CFP}_\Pi$ implies polynomial-time decidability in every world $\omega \in \Omega_n$.*

Proof. The definition of $\mathbf{CFP}_\Pi$ requires a uniform polynomial bound for every $\omega \in \Omega_n$. Hence every world-specific language L_ω is decided in polynomial time by its corresponding P_ω . $\square$

10 Counterfactual Debugging

Counterfactual debugging formalizes the question: what smallest change in the computational world would have prevented the observed failure?

Definition 14 (Failure predicate). *A failure predicate is a map*

$$F : \Omega \times X \rightarrow \{0, 1\},$$

where $F(\omega, x) = 1$ means that execution in world ω on input x fails.

Definition 15 (Minimal counterfactual repair). *Given actual failure $F(\omega_0, x_0) = 1$, a minimal counterfactual repair is*

$$\omega^* \in \arg \min_{\omega \in \Omega} d_\Omega(\omega, \omega_0) \quad \text{subject to} \quad F(\omega, x_\omega) = 0.$$

Theorem 4 (Existence on finite world graphs). *If Ω is finite and there exists at least one $\omega \in \Omega$ such that $F(\omega, x_\omega) = 0$, then a minimal counterfactual repair exists.*

Proof. The feasible set

$$\Omega_F = \{\omega \in \Omega : F(\omega, x_\omega) = 0\}$$

is nonempty and finite. The real-valued function

$$\omega \mapsto d_\Omega(\omega, \omega_0)$$

therefore attains a minimum on Ω_F . Any minimizer is a minimal counterfactual repair. $\square$

Algorithm 1 (Counterfactual debugging on a finite world graph). *Given a finite graph $G = (\Omega, E)$, actual world ω_0 , and failure predicate F :*

1. *Initialize a queue with ω_0 .*
2. *Mark ω_0 as visited.*
3. *While the queue is nonempty, remove the first world ω .*
4. *If $F(\omega, x_\omega) = 0$, return ω as a minimal repair.*
5. *Otherwise add all unvisited neighbors of ω to the queue.*
6. *If no feasible world is found, report that no repair exists inside the explored graph.*

Breadth-first search gives a minimal repair when graph distance is the world metric.

11 Counterfactual Security

Let S_ω be a system and A_ω an adversary in world ω .

Definition 16 (Counterfactual security). *A system bundle $\mathbf{S}$ is (ϵ, Ω_A) -counterfactually secure if*

$$\forall \omega \in \Omega_A, \quad \mathbb{P}[A_\omega(S_\omega) = 1] \leq \epsilon,$$

where $A_\omega(S_\omega) = 1$ denotes adversarial success.

Definition 17 (Security fragility). *The security fragility of $\mathbf{S}$ over Ω_A is*

$$\phi(\mathbf{S}) = \sup_{\omega \in \Omega_A} \mathbb{P}[A_\omega(S_\omega) = 1] - \mathbb{P}[A_{\omega_0}(S_{\omega_0}) = 1].$$

A positive $\phi(\mathbf{S})$ means that the actual world understates the system's exposure. A system may appear secure merely because the actual adversary was weak, unlucky, uninformed, or absent.

12 Counterfactual Statistics and Data Science

In data science, the observed dataset D is one realization from a larger space of possible datasets. Counterfactual Computer Science studies algorithms as functions of unrealized datasets:

$$A(D_\omega) = \theta_\omega.$$

Definition 18 (Counterfactual estimator). *A counterfactual estimator is a family*

$$\hat{\theta}_\omega = A_\omega(D_\omega), \quad \omega \in \Omega.$$

Its counterfactual bias is

$$B_{\text{CF}} = \sup_{\omega \in \Omega} \left\| \mathbb{E}_\omega[\hat{\theta}_\omega] - \theta_\omega \right\|.$$

Definition 19 (Counterfactual variance). *The counterfactual variance of an estimator family is*

$$V_{\text{CF}} = \mathbb{E}_{\omega \sim \mu} \left[\left\| \hat{\theta}_\omega - \mathbb{E}_{\omega' \sim \mu}[\hat{\theta}_{\omega'}] \right\|^2 \right].$$

Definition 20 (Counterfactual data value). *For a data point $z_i \in D$, define its counterfactual value by*

$$\text{Val}_{\text{CF}}(z_i) = \mathbb{E}_{\omega \sim \mu} [U(A_\omega(D_\omega)) - U(A_\omega(D_\omega \setminus \{z_i\}))],$$

where U is a utility, accuracy, welfare, or safety functional.

Statistical object	Classical version	Counterfactual version
Estimator	$\hat{\theta} = A(D)$	$\hat{\theta}_\omega = A_\omega(D_\omega)$
Bias	$\mathbb{E}[\hat{\theta}] - \theta$	$\sup_\omega \ \mathbb{E}_\omega[\hat{\theta}_\omega] - \theta_\omega\ $
Variance	$\mathbb{E}\ \hat{\theta} - \mathbb{E}\hat{\theta}\ ^2$	Variation across data-worlds ω
Influence	Effect of one observation	Effect of one observation across possible datasets
Generalization	Train/test gap	Worst-world or average-world train/test gap

Table 2: Classical and counterfactual statistical objects.

13 Counterfactual Machine Learning

Let D_ω be training data, R_ω a reward or loss function, E_ω a deployment environment, and M_ω a trained model. Then

$$M_\omega = \text{Train}_\omega(D_\omega, R_\omega, E_\omega).$$

Definition 21 (Counterfactual risk). *Given loss ℓ_ω , the risk of M_ω in world ω is*

$$\text{Risk}_\omega(M_\omega) = \mathbb{E}_{(x,y) \sim \mathcal{D}_\omega} [\ell_\omega(M_\omega(x), y)].$$

The worst-world counterfactual risk is

$$\text{Risk}_{\text{CF}}^{\max}(\mathbf{M}) = \sup_{\omega \in \Omega} \text{Risk}_\omega(M_\omega).$$

The average-world counterfactual risk is

$$\text{Risk}_{\text{CF}}^\mu(\mathbf{M}) = \mathbb{E}_{\omega \sim \mu} [\text{Risk}_\omega(M_\omega)].$$

Definition 22 (Counterfactual alignment). *An AI bundle $\mathbf{M}$ is (ϵ, Ω') -aligned with respect to a normative loss ℓ_ω^A if*

$$\sup_{\omega \in \Omega'} \mathbb{E}_{x \sim \mathcal{D}_\omega} [\ell_\omega^A(M_\omega(x))] \leq \epsilon.$$

Theorem 5 (Finite-world uniform generalization bound). *Let $\Omega = \{\omega_1, \dots, \omega_K\}$ be finite. Suppose for each ω , losses are bounded in $[0, 1]$, and n_ω independent samples are drawn from $\mathcal{D}_\omega$. Then with probability at least $1 - \delta$, for any fixed model bundle $\mathbf{M}$,*

$$\forall \omega \in \Omega, \quad \left| \text{Risk}_\omega(M_\omega) - \widehat{\text{Risk}}_\omega(M_\omega) \right| \leq \sqrt{\frac{\log(2K/\delta)}{2n_\omega}}.$$

Proof. For a fixed ω , Hoeffding's inequality gives

$$\mathbb{P} \left(\left| \text{Risk}_\omega(M_\omega) - \widehat{\text{Risk}}_\omega(M_\omega) \right| > t_\omega \right) \leq 2 \exp(-2n_\omega t_\omega^2).$$

Set

$$t_\omega = \sqrt{\frac{\log(2K/\delta)}{2n_\omega}}.$$

Then the failure probability for world ω is at most δ/K . A union bound over the K worlds gives total failure probability at most δ . Therefore the stated bound holds simultaneously for all worlds with probability at least $1 - \delta$. $\square$

14 Counterfactual Operating Systems and Distributed Systems

Operating systems and distributed systems are naturally counterfactual because the actual execution trace is one among many possible schedules.

Let an execution trace be

$$H = (e_1, e_2, \dots, e_T),$$

where each e_t is an event such as a process step, message delivery, cache miss, page fault, interrupt, lock acquisition, or node failure.

Definition 23 (Schedule counterfactual). *A schedule counterfactual is a trace*

$$H' = (e_{\pi(1)}, e_{\pi(2)}, \dots, e_{\pi(T)})$$

generated from the actual trace H by an admissible transformation π , subject to causal constraints.

Definition 24 (Trace stability). *A distributed protocol is trace-stable over a set $\mathcal{H}$ of counterfactual traces if*

$$\forall H, H' \in \mathcal{H}, \quad \text{Out}(H) = \text{Out}(H').$$

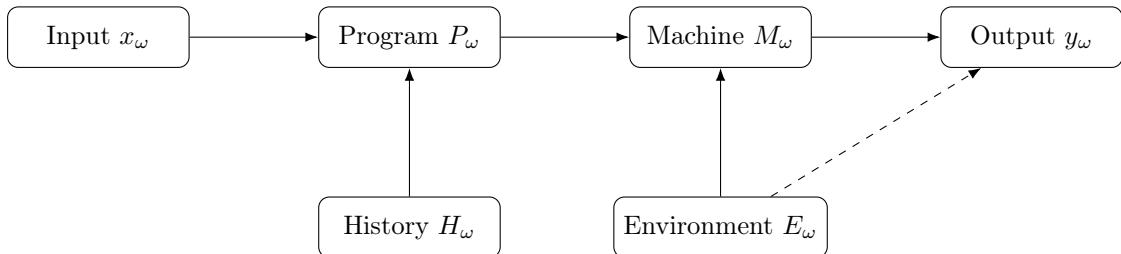


Figure 2: A world-indexed computational pipeline.

15 Counterfactual Programming Languages

A counterfactual programming language would make worlds first-class objects. A program could explicitly define a base world, perturbation operators, and comparisons across executions.

Algorithm 2 (Counterfactual execution schema). *Given a program P , input x , actual world ω_0 , perturbation operators $\Delta_1, \dots, \Delta_m$, and metric d_Ω :*

1. *Execute $P_{\omega_0}(x_{\omega_0})$ and store y_{ω_0} .*
2. *For each perturbation operator Δ_j , construct $\omega_j = \Delta_j(\omega_0)$.*
3. *Execute $P_{\omega_j}(x_{\omega_j})$ and store y_{ω_j} .*
4. *Compute differences $d_Y(y_{\omega_j}, y_{\omega_0})$.*
5. *Return the outcome profile*

$$(y_{\omega_0}, y_{\omega_1}, \dots, y_{\omega_m})$$

and its robustness summary.

16 A General Architecture for Counterfactual Analysis

A practical counterfactual analysis engine has four components:

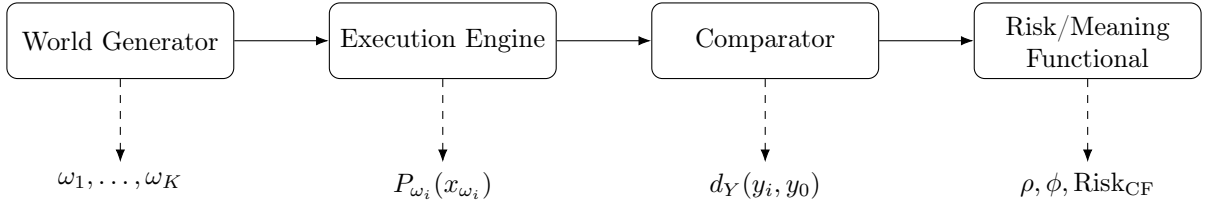


Figure 3: A generic architecture for Counterfactual Computer Science.

The following space was deliberately left blank.

17 Research Program

Counterfactual Computer Science suggests several research directions.

Subfield	Representative problems
Counterfactual algorithms	Design algorithms whose guarantees survive input, machine, and resource perturbations.
Counterfactual complexity	Classify languages by worst-world and distributional world complexity.
Counterfactual verification	Verify specifications across families of possible execution worlds.
Counterfactual debugging	Find minimal world-changes that prevent failure.
Counterfactual security	Measure security under unrealized adversaries and attack surfaces.
Counterfactual systems	Analyze alternate schedules, crashes, message delays, and memory states.
Counterfactual data science	Estimate bias, variance, and data value across possible datasets.
Counterfactual machine learning	Study training histories, deployment shifts, alignment, and robustness across worlds.
Counterfactual program- ming languages	Make possible-world execution and comparison first-class language constructs.

Table 3: A possible research map for Counterfactual Computer Science.

18 Limitations and Boundary Conditions

Counterfactual Computer Science requires careful restriction of the world space Ω . If Ω is too small, the analysis misses important alternatives. If Ω is too large, the analysis becomes vacuous, intractable, or metaphysically ungrounded. Therefore, each application must specify:

$$(\Omega, d_\Omega, \mu, \mathcal{N}_r(\omega_0), S_\omega, \ell_\omega).$$

The field should not claim that all counterfactuals are equally meaningful. Only structured, admissible, and domain-justified alternatives should enter the analysis.

19 Conclusion

Counterfactual Computer Science studies computation through actual and unrealized executions. Its central object is a bundle of computations indexed by possible worlds:

$$\mathbf{P} = \{P_\omega\}_{\omega \in \Omega}.$$

Its central measurements include counterfactual correctness, equivalence, robustness, debugging distance, security fragility, statistical stability, and machine-learning risk. Its central philosophical claim is that computation is not merely execution, but execution situated inside a neighborhood of alternative executions.

The slogan of the field is:

A computation is not only what it did; it is also what it would have done.

References

- [1] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 1936.
- [2] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 1969.
- [3] S. A. Cook. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971.
- [4] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, 1972.
- [5] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 1978.
- [6] L. G. Valiant. A theory of the learnable. *Communications of the ACM*, 1984.
- [7] J. Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2000.
- [8] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.
- [9] V. N. Vapnik. *Statistical Learning Theory*. Wiley, 1998.
- [10] S. Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.

Glossary

Actual world

The realized computational world ω_0 in which the observed execution occurred.

Computational world

A structured tuple containing inputs, outputs, programs, machines, resources, environments, histories, adversaries, and probability laws.

Counterfactual Computer Science

The proposed branch of computer science that studies computation across actual and unrealized computational worlds.

Counterfactual correctness

Correctness of a program bundle across a specified family of worlds.

Counterfactual debugging

The search for minimal changes to the computational world that would have prevented a failure.

Counterfactual equivalence

Equivalence of two program bundles across multiple worlds rather than only in the actual execution.

Counterfactual machine learning

The study of models under alternate training data, seeds, reward functions, deployment environments, and user populations.

Counterfactual neighborhood

The set of worlds within a specified distance from the actual world.

Counterfactual robustness

Stability of outputs under nearby world perturbations.

Counterfactual security

Security measured across possible adversarial worlds, not only the actual observed adversary.

Outcome profile

The vector or family of outputs produced by a program bundle across worlds.

Program bundle

A family $\{P_\omega\}_{\omega \in \Omega}$ of programs indexed by computational worlds.

Security fragility

The extent to which actual-world security understates risk in nearby adversarial worlds.

World distance

A metric, divergence, preorder, or graph distance used to compare computational worlds.

The End